Lab Report

of

Cryptography

Subject Code: CSC-327

Submitted To

SOCH COLLEGE OF IT

(AFFILIATED TO TRIBHUVAN UNIVERSITY)

Ranipauwa, Pokhara – 11

Submitted by:

Abhishek Thapa

University Registration number:5-2-1179-2-2023

College Roll Number: 02

Program: Bachelor of Science in Computer Science and Information Technology (B.Sc.CSIT)

Semester: 5th semester

List of Exercises

S/ N	Title of Experiment	Date	Remarks
1	**IMPLEMENT CEASER CIPHER ALGORITHM (C-PROGRAMMING).**	**2082-11-3**	
2	**IMPLEMENT HILL CIPHER ALGORITHM {C-PROGRAMMING).**	**2082-11-3**	
3	**IMPLEMENT RAIL FENCE ALOGRITHM(C-PROGRAMMING).**	**2082-11-3**	
4	**IMPLEMENT DIFFIE-HELLMAN ALGORITHM (C-PROGRAMMING).**	**2082-11-4**	
5	**IMPLEMENT RSA ALGORITHM (C-PROGRAMMING).**	**2082-11-4**	
6	**IMPLEMENT EUCLIDEAN ALGORITHM (C-PROGRAMMING).**	**2082-11-8**	
7	**IMPLEMENT MODULAR ARITHMETIC (C-PROGRAMMING).**	**2082-11-8**	
8	**IMPLEMENT MD4 HASH ALGORITHM (C-PROGRAMMING).**	**2082-11-12**	
9	**IMPLEMENT MD5 HASH ALGORITHM (C-PROGRAMMING).**	**2082-11-12**	
10			
11			
12			

LAB-1:

IMPLEMENT CEASER CIPHER ALGORITHM (C-PROGRAMMING).

THEORY:

- The earliest known, and the simplest, use of a substitution cipher was by Julius Caesar.
- The Caesar cipher involves replacing each letter of the alphabet with the let ter standing three places further down the alphabet.
- For example,
- plain: meet me after the toga party
- cipher: PHHW PH DIWHU WKH WRJD SDUWB
- Encryption: C = E(k,p) = (p + k) mod 26
- Decryption: p = D(k,C) = (C- k) mod 26

ALGORITHM:

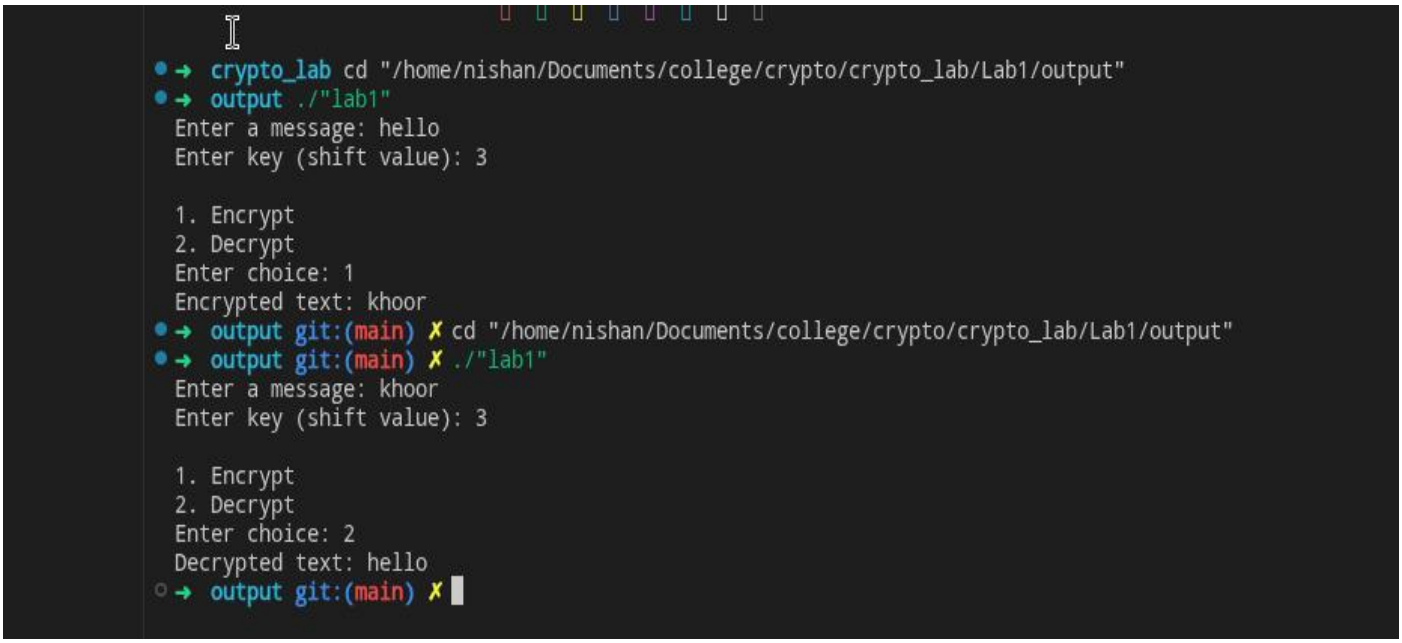
- Step 1: start
- Step 2: Conver a letter to its corresponding order as in aplhabet . Start form 0 to 25, A=0,
- Step 3: Compute Cipher text C = E(k,p) = (p + k) mod 26 where 0<=k<=25
- Step 4: Compute plain text from ciphertext as: D(k,C) = (C- k) mod 26

CODE:

```
#include <stdio.h>

#include <ctype.h>

void encrypt(char text[], int key);

void decrypt(char text[], int key);

int main() {

   char text[100];

   int key, choice;

    printf("Enter a message: ");    fgets(text, sizeof(text), stdin);

    printf("Enter key (shift value): ");  scanf("%d", &key);

   key = key % 26; // To handle large keys

   printf("\n1. Encrypt\n2. Decrypt\nEnter choice: ");    scanf("%d", &choice);

    if (choice == 1)

      encrypt(text, key);

   else if (choice == 2)

      decrypt(text, key);

   else
```

```
 printf("Invalid choice!");   return 0;
}void encrypt(char text[], int key) {
    int i;
    for (i = 0; text[i] != '\0'; i++) {
        if (isupper(text[i])) {
            text[i] = ((text[i] - 'A' + key) % 26) + 'A';
        }
        else if (islower(text[i])) {text[i] = ((text[i] - 'a' + key) % 26) + 'a';
        }
    } printf("Encrypted text: %s", text);
}void decrypt(char text[], int key) {
    int i;
    for (i = 0; text[i] != '\0'; i++) {
        if (isupper(text[i])) {
            text[i] = ((text[i] - 'A' - key + 26) % 26) + 'A';
        } else if (islower(text[i])) { text[i] = ((text[i] - 'a' - key + 26) % 26) + 'a';
        }
    }    printf("Decrypted text: %s", text);
}
```

OUTPUT:



LAB-2:

IMPLEMENT HILL CIPHER ALGORITHM (C-PROGRAMMING).

THEORY:

- The Hill Cipher is a polygraphic substitution cipher, which means it encrypts more than one letter at a time. It was invented by Lester S. Hill in 1929 and is based on matrix mathematics.
- Unlike simple ciphers that replace letters one by one, the Hill Cipher replaces a group of letters, making it more secure.
- The Hill Cipher uses:

1. Matrices
2. Modular arithmetic (mod 26)
3. Linear algebra

- Each letter of the alphabet is first converted into a number: A = 0, B = 1, C = 2, ..., Z = 25
- The key is a square matrix (2×2, 3×3, etc.)
- The size of the matrix decides how many letters are encrypted at once.
- The key matrix must be invertible mod 26
- Example Key (2×2 matrix):
- $K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$

ALGORITHM:

Encryption

1. Step 1:Prepare Plaintext
   - Remove spaces and symbols
   - Convert to uppercase
   - Divide plaintext into blocks of size n
   - Pad with X if required
2. Step 2: Convert Letters to Numbers
3. Step 3: Apply Encryption Formula ⍰=(⍰×⍰) mod 26

Decryption

Steps:

- Compute Determinant :det(K)
- Find Modular Inverse of Determinant: det(K)−1mod26
- Compute Inverse Key Matrix: K−1=det(K)−1×adj(K)mod26
- Convert Ciphertext to Numbers
- Matrix Multiplication :$P = K^{-1} \times C$
- Apply Modulo 26 :$P = (K^{-1} \times C) \bmod 26$
- Note: gcd(det(K),26)=1 for inverse to exist.

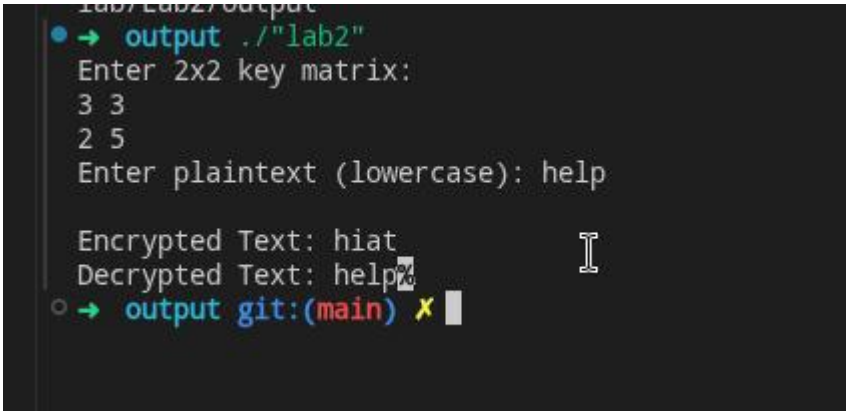
CODE:

```
#include <stdio.h>

#include <string.h>

// Find modular inverse of determinant

int modInverse(int det) {

   det = det % 26;

   for (int i = 1; i < 26; i++) {

      if ((det * i) % 26 == 1)

         return i;

   } return -1;
```

```
}
void encrypt(char msg[], int key[2][2], char cipher[]) {
   int len = strlen(msg);
   // Pad with 'x' if odd length
   if (len % 2 != 0) {
      msg[len] = 'x';
      msg[len + 1] = '\0';
      len++;
   }  for (int i = 0; i < len; i += 2) {
      int a = msg[i] - 'a';
      int b = msg[i+1] - 'a';
      int c1 = (key[0][0]*a + key[0][1]*b) % 26;
      int c2 = (key[1][0]*a + key[1][1]*b) % 26;
      cipher[i]   = c1 + 'a';
      cipher[i+1] = c2 + 'a';
   } cipher[len] = '\0';
}
void decrypt(char cipher[], int key[2][2], char plain[]) {
   int len = strlen(cipher);
   int invKey[2][2];
  // Determinant
   int det = (key[0][0]*key[1][1] - key[0][1]*key[1][0]) % 26;
   if (det < 0) det += 26;
   int detInv = modInverse(det);
   if (detInv == -1) {
      printf("\nKey matrix not invertible. Decryption not possible.\n");   return;
   }
   invKey[0][0] = ( key[1][1] * detInv) % 26;
   invKey[0][1] = (-key[0][1] * detInv) % 26;
   invKey[1][0] = (-key[1][0] * detInv) % 26;
   invKey[1][1] = ( key[0][0] * detInv) % 26;
```

```
    // Fix negatives
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)     if (invKey[i][j] < 0)
                invKey[i][j] += 26;
    // Decrypt
    for (int i = 0; i < len; i += 2) {
        int a = cipher[i] - 'a';
        int b = cipher[i+1] - 'a';
        int p1 = (invKey[0][0]*a + invKey[0][1]*b) % 26;
        int p2 = (invKey[1][0]*a + invKey[1][1]*b) % 26;
        plain[i]   = p1 + 'a';
        plain[i+1] = p2 + 'a';
    }   plain[len] = '\0';
}
int main() {  int key[2][2];
    char msg[100], cipher[100], plain[100];
    printf("Enter 2x2 key matrix:\n");
    for (int i = 0; i < 2; i++){
        for (int j = 0; j < 2; j++)  {     scanf("%d", &key[i][j]);} }
    printf("Enter plaintext (lowercase): ");    scanf("%s", msg);
    encrypt(msg, key, cipher);
    printf("\nEncrypted Text: %s", cipher);     decrypt(cipher, key, plain);
    printf("\nDecrypted Text: %s", plain);  return 0;
}
```

OUTPUT:



LAB-3:

IMPLEMENT RAIL FENCE ALOGRITHM(C-PROGRAMMING).

THEORY:

- It is multi letter encryption cipher invented in 1854 by Charles Wheatstone but named after Lord Playfair who promoted the use of cipher.
- It is based on the use of 5x5 matrix of letters constructed using keyword.
- The playfair cipher is relatively fast and does not require special equipment.
- British Forces used it for tactical purposes during Word War I.

Note:

1. 1. The matrix must be constructed using key letters form the start. 2. Any duplicate letter in key is removed.
2. 3. After filling the key letters fill other cells using letters of alphabet.
3. 4. Put I and J in same cells as 5x5 matrix only supports 25 letters.

ALGORITHM:

For Encryption:

- If both pair of letters are in same row, select the letter which belongs to the right side of it. (if cells end wrap around)
- If both pair of letters are in same column, select letter which belongs to the down side of it. (is cells end wrap around).
- If they form a box shape, select the letter opposite to it within that box. (must be in same row as the letter).

For Decryption: reverse the encryption process.

CODE:

```
#include <stdio.h>

#include <string.h>

void encryptRailFence(char text[], int key);

void decryptRailFence(char cipher[], int key);

int main() {

   char text[100];

   int key;

   printf("Enter text: ");
```

```
    fgets(text, sizeof(text), stdin);
    // Remove newline from fgets
    text[strcspn(text, "\n")] = '\0';
    printf("Enter key (number of rails): ");    scanf("%d", &key);
    encryptRailFence(text, key);
    return 0;
}
void encryptRailFence(char text[], int key) {
    int len = strlen(text);
    char rail[key][len];
    int i, j;
    for (i = 0; i < key; i++)
        for (j = 0; j < len; j++){     rail[i][j] = '\n';}
    int dir_down = 0, row = 0, col = 0;
    for (i = 0; i < len; i++) {
        if (row == 0 || row == key - 1){    dir_down = !dir_down;}
        rail[row][col++] = text[i];
        if (dir_down)       row++;
        else   row--;
    }
    char cipher[100];
    int index = 0;
    for (i = 0; i < key; i++)
        for (j = 0; j < len; j++)
            if (rail[i][j] != '\n')
                cipher[index++] = rail[i][j];
    cipher[index] = '\0';
    printf("\nCipher Text: %s\n", cipher);   decryptRailFence(cipher, key);
}
void decryptRailFence(char cipher[], int key) {
    int len = strlen(cipher);
```

```
    char rail[key][len];
    int i, j;
    for (i = 0; i < key; i++)
        for (j = 0; j < len; j++){   rail[i][j] = '\n';}
    int dir_down, row = 0, col = 0;
    for (i = 0; i < len; i++) {
        if (row == 0) dir_down = 1;
        if (row == key - 1) dir_down = 0;
        rail[row][col++] = '*';
        if (dir_down)   row++;
        else   row--;
    }
    int index = 0;
    for (i = 0; i < key; i++)
        for (j = 0; j < len; j++)
            if (rail[i][j] == '*')
                rail[i][j] = cipher[index++];
    char result[100];
    row = 0; col = 0;
    int k = 0;
    for (i = 0; i < len; i++) {
        if (row == 0) dir_down = 1;
        if (row == key - 1) dir_down = 0;
        result[k++] = rail[row][col++];
        if (dir_down)      row++;
        else     row--;
    }
    result[k] = '\0';     printf("Decrypted Text: %s\n", result);
}
```

OUTPUT:

LAB-4:

IMPLEMENT DIFFIE-HELLMAN ALGORITHM (C-PROGRAMMING).

THEORY:

- The Diffie-Hellman (DH) algorithm is a key-exchange protocol that allows two parties to establish a shared secret over an insecure channel without transmitting the secret itself.
- It was proposed by Whitfield Diffie and Martin Hellman in 1976 and forms the basis of many modern cryptographic systems.
- The security of Diffie-Hellman relies on the Discrete Logarithm Problem: given p, g, and g^x mod p, it is computationally infeasible to find x for large primes.
- Both parties agree on public values: a large prime p and a primitive root g (generator).
- Each party selects a secret private key and computes a public key using modular exponentiation.
- The shared secret is computed independently by each party and yields the same value: K = g^(ab) mod p.

ALGORITHM:

- Step 1: Start.
- Step 2: Agree on public values - a large prime p and a primitive root g.
- Step 3: Party A chooses private key a, computes public key A = g^a mod p, and sends A to B.
- Step 4: Party B chooses private key b, computes public key B = g^b mod p, and sends B to A.
- Step 5: Party A computes shared secret: Key = B^a mod p.
- Step 6: Party B computes shared secret: Key = A^b mod p.
- Step 7: Both shared secrets are equal: Key = g^(ab) mod p. Stop.

CODE:

```
#include <stdio.h>

long long power(long long base, long long exp, long long mod) {

   long long result = 1;

   base = base % mod;
```

```
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        exp = exp / 2;
        base = (base * base) % mod;
    }
    return result;
}

int main() {    long long p, g, a, b, A, B, keyA, keyB;
    printf("Enter public prime number (p): ");    scanf("%lld", &p);
    printf("Enter primitive root / generator (g): ");    scanf("%lld", &g);
    printf("Enter Party A private key (a): ");  scanf("%lld", &a);
    printf("Enter Party B private key (b): ");    scanf("%lld", &b);
    A = power(g, a, p);   // A = g^a mod p
    B = power(g, b, p);   // B = g^b mod p
    printf("\nParty A public key = %lld\n", A);
    printf("Party B public key = %lld\n", B);
    keyA = power(B, a, p);  // A computes shared secret
    keyB = power(A, b, p);  // B computes shared secret

    printf("Shared Key computed by A = %lld\n", keyA);
    printf("Shared Key computed by B = %lld\n", keyB);

    if (keyA == keyB)
        printf("Key Exchange Successful! Shared Secret = %lld\n", keyA);
    else
        printf("Key Exchange Failed!\n");

    return 0;
}
```

OUTPUT:

```
→ output git:(main) ✗ ./"lab4"
Enter public prime number (p): 23
Enter primitive root / generator (g): 5
Enter Party A private key (a): 6
Enter Party B private key (b): 15

Party A public key = 8
Party B public key = 19
Shared Key computed by A = 2
Shared Key computed by B = 2
Key Exchange Successful! Shared Secret = 2
```

LAB-5:

IMPLEMENT RSA ALGORITHM (C-PROGRAMMING).

THEORY:

- RSA (Rivest-Shamir-Adleman) is a widely used public-key cryptosystem invented in 1977. It enables secure data transmission and digital signatures.
- RSA is based on the mathematical difficulty of factoring the product of two large prime numbers (the Integer Factorization Problem).
- RSA uses two keys: a Public Key for encryption (shared openly) and a Private Key for decryption (kept secret).
- Key Generation: Choose two large primes p and q. Compute n = p*q (modulus) and phi(n) = (p-1)*(q-1).
- Choose e such that 1 < e < phi(n) and gcd(e, phi(n)) = 1. Find d such that (d*e) mod phi(n) = 1.
- Public Key = (e, n). Private Key = (d, n).
- Encryption: C = M^e mod n. Decryption: M = C^d mod n.

ALGORITHM:

- Step 1: Start.
- Step 2: Select two large prime numbers p and q.
- Step 3: Compute n = p * q and phi(n) = (p-1) * (q-1).
- Step 4: Choose e such that 1 < e < phi(n) and gcd(e, phi(n)) = 1.
- Step 5: Find d (private key) such that (d * e) mod phi(n) = 1.
- Step 6: Public Key = (e, n), Private Key = (d, n).
- Step 7: Encryption - Ciphertext C = M^e mod n.
- Step 8: Decryption - Plaintext M = C^d mod n.
- Step 9: Stop.

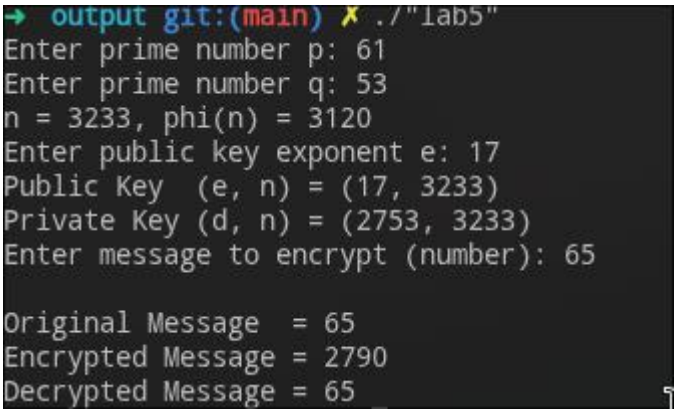
CODE:

```
#include <stdio.h>

int gcd(int a, int b) {
```

```
    while (b != 0) {       int t = b; b = a % b; a = t;
    }   return a;
}
long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {       if (exp % 2 == 1)
          result = (result * base) % mod;
        exp /= 2;
        base = (base * base) % mod;
    }    return result;
}
int main() {    int p, q, e;
    long long msg;
    printf("Enter prime number p: ");    scanf("%d", &p);
    printf("Enter prime number q: ");    scanf("%d", &q);
    int n   = p * q;
    int phi = (p - 1) * (q - 1);
    printf("n = %d, phi(n) = %d\n", n, phi);
    printf("Enter public key exponent e: ");    scanf("%d", &e);
    if (gcd(e, phi) != 1) {     printf("Invalid e! gcd(e, phi) must be 1.\n");
       return 1;
    }
    int d = 1;
    while ((d * e) % phi != 1) d++;
    printf("Public Key  (e, n) = (%d, %d)\n", e, n);
    printf("Private Key (d, n) = (%d, %d)\n", d, n);
    printf("Enter message to encrypt (number): ");    scanf("%lld", &msg);
    long long cipher    = power(msg, e, n);
    long long decrypted = power(cipher, d, n);
    printf("\nOriginal Message  = %lld\n", msg);
    printf("Encrypted Message = %lld\n", cipher);
    printf("Decrypted Message = %lld\n", decrypted);
```

```
    return 0;

}
```

OUTPUT:



LAB-6:

IMPLEMENT EUCLIDEAN ALGORITHM (C-PROGRAMMING).

THEORY:

- The Euclidean Algorithm is one of the oldest known algorithms, dating back to ancient Greece (around 300 BC), attributed to the mathematician Euclid.
- It is used to compute the Greatest Common Divisor (GCD) of two integers.
- The GCD of two numbers is the largest positive integer that divides both numbers without a remainder.
- The algorithm is based on the principle: GCD(a, b) = GCD(b, a mod b), and terminates when b = 0.
- It is widely used in cryptography for key generation (RSA), modular inverse computation, and number theory.

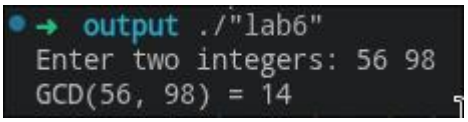
ALGORITHM:

- Step 1: Start.
- Step 2: Input two integers a and b.
- Step 3: If b == 0, then GCD = a. Stop.
- Step 4: Compute remainder r = a mod b.
- Step 5: Set a = b and b = r.
- Step 6: Go to Step 3.
- Step 7: Output GCD and Stop.

CODE:

```
#include <stdio.h>

int gcd(int a, int b) {

   while (b != 0) {    int r = a % b;

      a = b;

      b = r;

   }    return a;

}

int main() {    int a, b;

   printf("Enter two integers: ");    scanf("%d %d", &a, &b);

   printf("GCD(%d, %d) = %d\n", a, b, gcd(a, b));

   return 0;

}
```

OUTPUT:



LAB-7:

IMPLEMENT MODULAR ARITHMETIC (C-PROGRAMMING).

THEORY:

- Modular arithmetic is a system of arithmetic for integers where numbers 'wrap around' upon reaching a certain value called the modulus.
- It was formally introduced by Carl Friedrich Gauss in 1801 in his work Disquisitiones Arithmeticae.
- In modular arithmetic, a mod n gives the remainder when a is divided by n.
- It is fundamental in cryptography: used in RSA, Diffie-Hellman, and many other algorithms.
- Key operations include: modular addition, subtraction, multiplication, exponentiation, and finding modular inverses.
- Modular inverse of a (mod n) exists if and only if gcd(a, n) = 1 (i.e., a and n are coprime).

ALGORITHM:

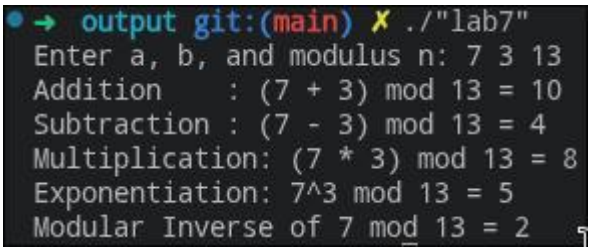
- Step 1: Start.
- Step 2: Input values a, b, and modulus n.
- Step 3: Compute addition: result = (a + b) mod n.
- Step 4: Compute subtraction: result = ((a - b) % n + n) % n.
- Step 5: Compute multiplication: result = (a * b) mod n.
- Step 6: Compute modular exponentiation: result = (a^b) mod n using fast exponentiation.
- Step 7: Find modular inverse of a mod n using extended Euclidean algorithm if gcd(a,n)=1.
- Step 8: Output all results and Stop.

CODE:

```
#include <stdio.h>


long long mod_exp(long long base, long long exp, long long mod) {
   long long result = 1;
   base = base % mod;
   while (exp > 0) {
      if (exp % 2 == 1)
         result = (result * base) % mod;
      exp = exp / 2;
      base = (base * base) % mod;
   }
   return result;
}
```

```
int mod_inverse(int a, int n) {
    for (int x = 1; x < n; x++)
        if ((a * x) % n == 1)
            return x;
    return -1;
}

int main() {
    int a, b, n;
    printf("Enter a, b, and modulus n: ");
    scanf("%d %d %d", &a, &b, &n);
    printf("Addition    : (%d + %d) mod %d = %d\n", a, b, n, (a + b) % n);
    printf("Subtraction : (%d - %d) mod %d = %d\n", a, b, n, ((a - b) % n + n) % n);
    printf("Multiplication: (%d * %d) mod %d = %d\n", a, b, n, (a * b) % n);
    printf("Exponentiation: %d^%d mod %d = %lld\n", a, b, n, mod_exp(a, b, n));
    int inv = mod_inverse(a, n);
    if (inv != -1)
        printf("Modular Inverse of %d mod %d = %d\n", a, n, inv);
    else
        printf("Modular Inverse does not exist.\n");
    return 0;7
}
```

OUTPUT:



LAB-8:

IMPLEMENT MD4 HASH ALGORITHM (C-PROGRAMMING).

THEORY:

- MD4 (Message Digest 4) is a cryptographic hash function designed by Ronald Rivest in 1990.
- It produces a 128-bit (16-byte) hash value, typically expressed as a 32-character hexadecimal number.
- MD4 was designed for speed and efficiency on 32-bit machines.
- It operates on 512-bit blocks and uses three rounds of processing with different Boolean functions.
- MD4 is considered cryptographically broken and is no longer recommended for security-critical applications.
- It served as the basis for later algorithms including MD5 and the SHA family.
- MD4 uses three auxiliary functions: F(X,Y,Z) = (X AND Y) OR (NOT X AND Z), G(X,Y,Z) = (X AND Y) OR (X AND Z) OR (Y AND Z), H(X,Y,Z) = X XOR Y XOR Z.

ALGORITHM:

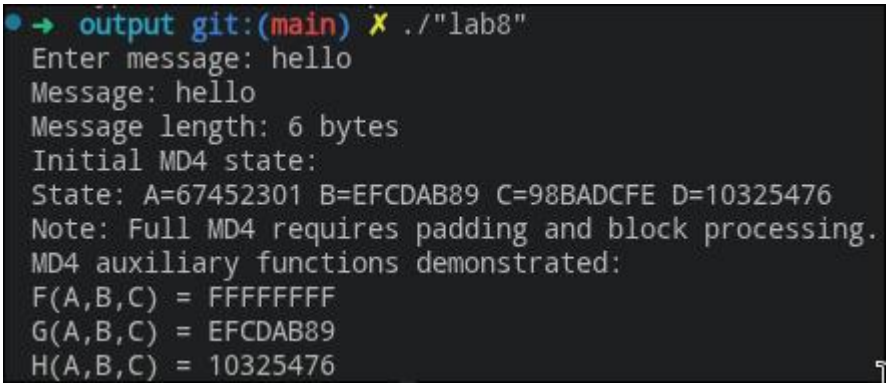
- Step 1: Start. Append padding bits to the input message so its length is 448 mod 512.
- Step 2: Append a 64-bit representation of the original message length.
- Step 3: Initialize MD buffer with four 32-bit words: A=0x67452301, B=0xEFCDAB89, C=0x98BADCFE, D=0x10325476.
- Step 4: Process each 512-bit block through three rounds.
- Step 5: Round 1 - 16 operations using function F and constant 0x00000000.
- Step 6: Round 2 - 16 operations using function G and constant 0x5A827999.
- Step 7: Round 3 - 16 operations using function H and constant 0x6ED9EBA1.
- Step 8: Add the output of each block to the buffer values.
- Step 9: Output the final 128-bit hash. Stop.

CODE:

```
#include <stdio.h>

#include <string.h>

#include <stdint.h>


/* Simplified MD4 demonstration - not full implementation */


#define F(x,y,z) (((x)&(y)) | ((~x)&(z)))

#define G(x,y,z) (((x)&(y)) | ((x)&(z)) | ((y)&(z)))

#define H(x,y,z) ((x)^(y)^(z))

#define ROTL(x,n) (((x)<<(n)) | ((x)>>(32-(n))))


void md4_print_state(uint32_t A, uint32_t B, uint32_t C, uint32_t D) {
```

```
    printf("State: A=%08X B=%08X C=%08X D=%08X\n", A, B, C, D);
}

int main() {
    char msg[256];
    printf("Enter message: ");
    fgets(msg, sizeof(msg), stdin);
    int len = strlen(msg);
    printf("Message: %s", msg);
    printf("Message length: %d bytes\n", len);
    /* Initialize MD4 state */
    uint32_t A = 0x67452301;
    uint32_t B = 0xEFCDAB89;
    uint32_t C = 0x98BADCFE;
    uint32_t D = 0x10325476;
    printf("Initial MD4 state:\n");
    md4_print_state(A, B, C, D);
    printf("Note: Full MD4 requires padding and block processing.\n");
    printf("MD4 auxiliary functions demonstrated:\n");
    printf("F(A,B,C) = %08X\n", F(A,B,C));
    printf("G(A,B,C) = %08X\n", G(A,B,C));
    printf("H(A,B,C) = %08X\n", H(A,B,C));
    return 0;
}
```

OUTPUT:



LAB-9:

IMPLEMENT MD5 HASH ALGORITHM (C-PROGRAMMING).

THEORY:

- MD5 (Message Digest 5) is a widely used cryptographic hash function designed by Ronald Rivest in 1991.
- It produces a 128-bit (16-byte) hash value, typically represented as a 32-character hexadecimal string.
- MD5 improved upon MD4 by adding a fourth round of processing and more complex operations.
- It processes input in 512-bit blocks, and uses four rounds of 16 operations each (64 total operations).
- MD5 uses four auxiliary functions: F, G, H, and I operating on 32-bit words.
- Each step uses a 64-element table T[i] = abs(sin(i+1)) * 2^32 for constants.
- Although widely used historically, MD5 is considered cryptographically broken due to collision vulnerabilities.
- Applications include file integrity checking and checksums (not for password storage or digital signatures).

ALGORITHM:

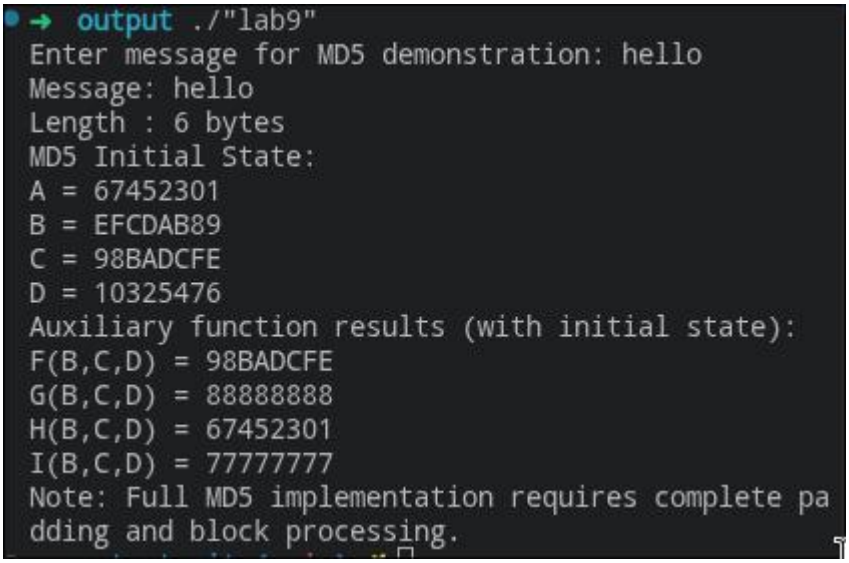
- Step 1: Start. Pad the input message so its length is 448 bits mod 512.
- Step 2: Append a 64-bit representation of the original message length.
- Step 3: Initialize four 32-bit words: A=0x67452301, B=0xEFCDAB89, C=0x98BADCFE, D=0x10325476.
- Step 4: Process each 512-bit block through 4 rounds of 16 operations each.
- Step 5: Round 1 - Uses F(B,C,D) = (B AND C) OR (NOT B AND D).
- Step 6: Round 2 - Uses G(B,C,D) = (B AND D) OR (C AND NOT D).
- Step 7: Round 3 - Uses H(B,C,D) = B XOR C XOR D.
- Step 8: Round 4 - Uses I(B,C,D) = C XOR (B OR NOT D).
- Step 9: Add the block output to the running hash values.
- Step 10: Output the final 128-bit digest as a 32-character hex string. Stop.

CODE:

```
#include <stdio.h>

#include <string.h>

#include <stdint.h>

#include <math.h>

#define F(b,c,d) (((b)&(c)) | ((~b)&(d)))

#define G(b,c,d) (((b)&(d)) | ((c)&(~d)))

#define H(b,c,d) ((b)^(c)^(d))

#define I(b,c,d) ((c)^((b)|(~d)))

#define ROTL(x,n) (((x)<<(n))|((x)>>(32-(n))))

void md5_demo(const char *msg) {

   uint32_t A = 0x67452301;
```

```
    uint32_t B = 0xEFCDAB89;
    uint32_t C = 0x98BADCFE;
    uint32_t D = 0x10325476;
    printf("MD5 Initial State:\n");
    printf("A = %08X\n", A);
    printf("B = %08X\n", B);
    printf("C = %08X\n", C);
    printf("D = %08X\n", D);
    printf("Auxiliary function results (with initial state):\n");
    printf("F(B,C,D) = %08X\n", F(B,C,D));
    printf("G(B,C,D) = %08X\n", G(B,C,D));
    printf("H(B,C,D) = %08X\n", H(B,C,D));
    printf("I(B,C,D) = %08X\n", I(B,C,D));
}
int main() {
    char msg[256];
    printf("Enter message for MD5 demonstration: ");   fgets(msg, sizeof(msg), stdin);
    printf("Message: %s", msg);
    printf("Length : %lu bytes\n", strlen(msg));
    md5_demo(msg);
    printf("Note: Full MD5 implementation requires complete padding and block processing.\n");
    return 0;
}
```

OUTPUT:


➜ output ./"lab9"
Enter message for MD5 demonstration: hello
Message: hello
Length : 6 bytes
MD5 Initial State:
A = 67452301
B = EFCDAB89
C = 98BADCFE
D = 10325476
Auxiliary function results (with initial state):
F(B,C,D) = 98BADCFE
G(B,C,D) = 88888888
H(B,C,D) = 67452301
I(B,C,D) = 77777777
Note: Full MD5 implementation requires complete pa
dding and block processing.